

No-Excuses: A Multimodal Web Application for Hands-Free, Eyes-Free Workout Tracking

Crea Michelangelo
1993024

Gautieri Alessandro
2041850

Abstract

Workout tracking remains a disruptive element in physical exercise, forcing athletes to break their focus to interact with a smartphone screen or to purchase expensive wearable devices. This report introduces *No-Excuses*, a fully client-side, multimodal Progressive Web Application (PWA) designed to automate fitness tracking through seamless, hands-free, and eyes-free interactions. The system fuses four distinct interaction modalities: (1) **inertial sensor tracking** via accelerometer and gyroscope for pocket-mode exercise counting; (2) **computer vision** via Google MediaPipe Pose for camera-based repetition counting; (3) **voice command navigation** using the Web Speech API for controlling workout routines without touching the screen; and (4) **audio-haptic state communication** via the Web Audio API and device vibration to relay session state to the user without any visual attention required. We describe the system architecture, the state-machine algorithms developed for each modality, and the engineering solutions to significant real-world challenges: false positives caused by post-exercise device handling (solved with a real-time gravity orientation filter), erratic repetition detection due to sensor noise (solved with a sensor fusion energy metric), and iOS audio policy restrictions (solved with lazy AudioContext initialization). A comparative evaluation against a naive threshold baseline validates our approach.

1 Problem Statement

The core challenge addressed in this work is **multimodal activity recognition and repetition counting** in unconstrained, real-world environments using off-the-shelf mobile hardware. The objective is to accurately identify the concentric and eccentric phases of physical exercises, tally them in real-time, and communicate the system state to the user—all without requiring any direct visual or physical interaction with the device during execution.

During intense exercises such as pushups or pullups, athletes cannot safely or comfortably interact with a touchscreen. Traditional fitness tracking applications demand manual input between sets, breaking focus and inaccurately inflating rest times. While smartwatch-based systems provide partial automation, they require proprietary hardware. The challenge is therefore to build a complete tracking solution that works on a standard smartphone, is accessible through a browser, and preserves user privacy by processing all data on-device.

1.1 Application Scope and Real-World Use Cases

No-Excuses is a complete workout management platform with five major functional areas: (1) a workout **routine builder** supporting complex exercise structures such as Supersets, EMOM (Every Minute On the Minute), Isometric holds, and Pyramids; (2) an **active workout runner** with guided exercise progression, timers, and rest countdowns; (3) a **standalone repetition counter** supporting both camera and pocket (inertial) modes; (4) a **workout history** module backed by Supabase that logs every completed session; and (5) a session **resume checkpoint** system that saves progress to localStorage and prompts the user to continue after an app restart.

Real-world downstream applications include

personal fitness in home or outdoor settings (where no dedicated equipment exists), rehabilitation monitoring (where precise movement counts are medically relevant), and gym hygiene-conscious environments where users prefer not to touch their devices.

2 Need Finding

To validate our initial problem framing and identify the most impactful design requirements, we administered an online questionnaire to a group of 28 participants who regularly engage in calisthenics or resistance training (at least twice per week). The questionnaire focused on current tracking habits, friction points, and device usage during exercise.

2.1 Questionnaire Results

The key findings are summarized below.

Why athletes do not track repetitions in real time.

- **57.1%** — Inconvenient or impossible to touch the phone during a set (sweaty hands, mid-exercise)
- **21.4%** — Lose count during fatiguing or high-rep sets
- **14.3%** — Do not want to interrupt training flow between sets
- **7.1%** — Do not own a smartwatch

Would audio feedback replace the need to look at the screen?.

- **78.6%** — Yes, hearing the count announced aloud would be sufficient
- **14.3%** — Only if combined with haptic vibration
- **7.1%** — No, they want to see the screen

Interest in automatic repetition counting (no manual input).

- **89.3%** — Would use it if reliable and requiring no additional hardware

2.2 Identified Needs

Based on questionnaire responses and subsequent structured interviews, we identified three primary user needs that shaped the design of *No-Excuses*:

1. **Hands-free tracking:** The system must count repetitions autonomously. The user should never be required to touch the device during a set.
2. **Eyes-free state awareness:** The user must be able to understand the full state of the application (session active, rep counted, target reached, rest started) exclusively through audio and haptic signals—without looking at the screen.
3. **Voice-driven session control:** Navigating between exercises and controlling the workout timer must be achievable through spoken commands, eliminating the need to interact with the UI between sets.

These three needs directly map to the four interaction channels described in Section 5: inertial tracking and computer vision address Need 1, audio-haptic feedback addresses Need 2, and voice commands address Need 3.

3 Related Work

The state-of-the-art in digital fitness tracking is fragmented across three paradigms. **Manual-entry apps** such as MyFitnessPal or Strong rely on user compliance for post-set data entry, which consistently fails during high-intensity intervals. **Wearable IMU systems** (Apple Watch, Garmin) provide automatic detection but require proprietary, expensive hardware and are frequently avoided during heavy lifting due to wrist discomfort. **Camera-based systems** such as the Peloton Guide or PoseNet-based apps track pose landmarks effectively but demand fixed camera placement, specific lighting, and continuous user framing—conditions that are impractical in most real workout environments.

No-Excuses bridges these paradigms by offering a hybrid approach: inertial tracking for exercises where the phone is pocketed (no framing needed), computer vision for exercises where the phone can be propped up, and voice control to eliminate the need to ever touch the screen. Crucially, all processing runs client-side in the

browser with no external APIs, ensuring low latency and complete data privacy. No comparable open-source or commercial browser-based hybrid solution was found in the literature.

4 System Architecture

No-Excuses is built as a React and TypeScript Progressive Web App using the Vite build tool. The architecture is strictly client-side: no video frames, sensor readings, or user motion data are transmitted to external servers. Authentication and workout history persistence are handled via Supabase, which exclusively stores structured meta-data (exercise names, sets, timestamps) and never receives raw biometric data.

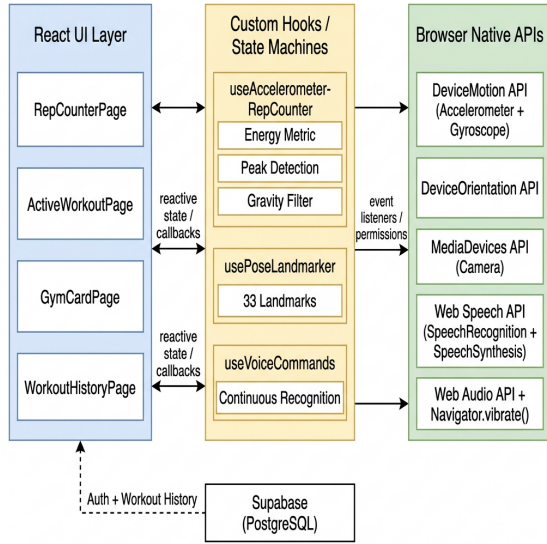


Figure 1: System architecture. The React UI layer communicates exclusively with Custom Hooks (state machines). Each hook interfaces with a specific browser API. All sensor and camera data remains on the device.

The core of the system is a set of decoupled React Hooks, each acting as a self-contained state machine:

- `useAccelerometerRepCounter` — manages the full lifecycle of an inertial tracking session (permission request, calibration countdown, motion energy computation, gravity filtering, repetition counting, and session termination).
- `usePoseLandmarker` — wraps the MediaPipe Pose WebAssembly model, providing frame-by-frame landmark data to the camera-based counting logic.

- `useVoiceCommands` — manages a persistent Web Speech API recognition session, mapping Italian spoken commands to workout control callbacks.

This separation of concerns allows each modality to be developed, tested, and tuned independently. The UI layer never contains algorithmic logic; it only renders reactive state from the hooks.

5 Multimodal Interaction Design

The multimodal design of *No-Excuses* was driven by a single guiding principle: *the user should never be forced to look at or touch the phone during exercise*. Four interaction channels are integrated to achieve this.

5.1 Channel 1: Inertial Sensor Tracking (Pocket Mode)

The first and primary interaction channel uses the smartphone’s accelerometer and gyroscope, accessed via the browser’s `DeviceMotionEvent` API. The phone is placed in the user’s pocket; the app infers each repetition from the rhythmic motion of the body.

5.1.1 Sensor Fusion Energy Metric

Raw accelerometer data contains gravity as a DC bias that must be removed. We apply an exponential low-pass filter with coefficient $\alpha_G = 0.95$ to continuously estimate the gravity vector $\mathbf{g}_t$:

$$\mathbf{g}_t = \alpha_G \mathbf{g}_{t-1} + (1 - \alpha_G) \mathbf{a}_t^{\text{raw}} \quad (1)$$

where $\mathbf{a}_t^{\text{raw}} = (a_x, a_y, a_z)$ is the raw accelerometer reading at time t . The *linear acceleration magnitude* is then:

$$L_t = \|\mathbf{a}_t^{\text{raw}} - \mathbf{g}_t\|_2 \quad (2)$$

A critical observation from our calibration data was that linear acceleration alone was insufficient to distinguish genuine exercise motion from pocket jitter, particularly for pushups. Pushups involve a strong rotational component of the thigh—the gyroscope captures this well. We therefore define the **Motion Energy** as a sensor-fused scalar:

$$E_t^{\text{raw}} = \omega_t + 40 \cdot L_t \quad (3)$$

where $\omega_t = \|(\dot{\alpha}_t, \dot{\beta}_t, \dot{\gamma}_t)\|_2$ is the gyroscope magnitude. The coefficient 40 was empirically deter-

mined during calibration to equalize the contribution of rotational and translational motion for calisthenics exercises. A final exponential smoothing pass with coefficient $\alpha_E = 0.4$ removes high-frequency noise:

$$E_t = 0.6 \cdot E_{t-1} + 0.4 \cdot E_t^{\text{raw}} \quad (4)$$

5.1.2 Peak-Count Algorithm for Pushups

Calibration revealed that for pushups the motion energy does not fall cleanly to baseline between consecutive repetitions when performed at high tempo. The classic dual-burst (up/down) detection therefore systematically missed repetitions. We implemented a **peak-count algorithm**:

1. A peak is *rising* when $E_t > \tau_{\text{active}}$ (threshold $\tau_{\text{active}} = 120$).
2. A peak is *confirmed* when the running maximum E_{max} exceeds $\rho \cdot \tau_{\text{active}}$ (peak ratio $\rho = 3$, i.e., 360).
3. A peak is *completed* when E_t drops back below τ_{active} , indicating the movement has concluded.
4. A repetition is *counted* when a completed, confirmed peak passes both the gravity orientation filter (Section 5.1.4) and the cooldown guard (Section 5.1.5).

For pullups, the dual-burst mode (single upward burst = one repetition) is used instead, as the signal is cleaner when the phone is worn on the body at waist level.

5.1.3 Calibration Dataset

To tune all numerical thresholds, we conducted 20 to 30 dedicated in-the-wild calibration sessions. Each session consisted of sets of 1, 5, and 10 repetitions performed at slow, normal, and rapid pacing. For each burst the system logged: `burstDurationMs`, `maxEnergy`, `minEnergy`, `avgEnergy`, `maxGyro`, and `gravityVec`. Analysis of these logs revealed:

- Genuine pushup peaks: $E_{\text{max}} \in [500, 1100]$, duration $\in [300, 2000]$ ms.
- Resting energy between repetitions: $E \approx 75\text{--}120$.
- Phone extraction from pocket: $E_{\text{max}} > 2000$, but orientation deviates $> 66^\circ$.

- Average human rep duration: 1.5–3.5 s; rapid reps as short as 1.2 s.

These measurements directly informed the per-exercise configuration:

Parameter	Pushups	Pullups
Algorithm mode	<i>peak_count</i>	<i>single_burst</i>
τ_{active}	120	140
τ_{rest}	75	78
τ_{gyro} (shake limit)	1200	673
τ_{dur} (min. duration, ms)	300	210
Cooldown (ms)	1200	500
Peak ratio ρ	3	—

Table 1: Per-exercise algorithm configuration derived from calibration data.

5.1.4 Gravity Orientation Filter

Problem. The most critical bug encountered during field testing was the generation of *false positives at session end*. When the user completed a set and pulled the phone out of their pocket, the extraction motion generated an energy spike $E > 2000$ —far above any genuine repetition. The naive algorithm counted this as 1–2 additional repetitions.

Solution. During the final second of the 5-second pre-workout countdown (when the phone is guaranteed to be stationary in the user’s pocket), the app captures a *baseline gravity vector*:

$$\mathbf{v}_{\text{base}} = \mathbf{g}_{T_0} \quad (5)$$

At every subsequent motion sample, the angular deviation between the current gravity direction and the baseline is computed as:

$$\theta = \arccos\left(\frac{\mathbf{v}_{\text{base}} \cdot \mathbf{g}_t}{\|\mathbf{v}_{\text{base}}\| \cdot \|\mathbf{g}_t\|}\right) \quad (6)$$

If $\theta > \theta_{\text{max}}$, the motion sample is discarded. We set $\theta_{\text{max}} = 66^\circ$ (≈ 1.15 rad). This value was chosen empirically:

- 45° was too strict: it rejected genuine pushup peaks when the phone shifted inside a loose pocket.
- 66° tolerates the natural pocket bounce during pushups while blocking the $\approx 90^\circ$ gravity shift that occurs when the user stands up from a horizontal prone position.

In all post-implementation field tests, this filter resulted in **zero false positives** during device extraction.

5.1.5 Repetition Cooldown Guard

Problem. Physical movements contain sub-peaks: brief secondary energy spikes at the bottom of a pushup (the elastic rebound). Without a minimum inter-repetition delay, these sub-peaks are double-counted.

Problem (opposite direction). A cooldown that is too long causes the system to ignore rapid repetitions performed in under 2 seconds.

Solution. A repetition is counted only if the elapsed time since the last counted repetition exceeds `repCooldownMs`. Based on calibration data showing rapid reps as short as 1.2 s, we set this value to **1200 ms** for pushups. This is the lowest safe value that prevents double-counting sub-peaks while capturing all realistic human rep tempos.

5.2 Channel 2: Computer Vision (Stand Mode)

For exercises where placing the phone in a pocket is impractical (e.g., pullups at a bar), the user props the device against a wall or water bottle so the front-facing camera frames their body. The app uses **Google MediaPipe Pose** (1), running entirely via WebAssembly inside the browser, to extract 33 3D anatomical landmarks per frame in real-time.

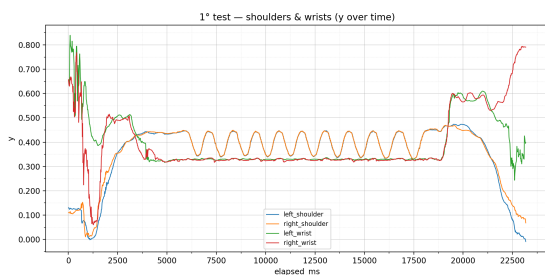


Figure 2: MediaPipe landmark time-series for a pullup session (23.5 s). Y-axis is the normalized vertical coordinate (0 = top of frame, 1 = bottom). Three distinct phases are visible. **Phase 1 (0–2.5 s):** setup noise—the user is positioning the phone; all four landmarks are unstable and scattered across the full Y-range. **Phase 2 (2.5–19.5 s):** active exercise—the shoulder landmarks (blue/orange) oscillate rhythmically between $y \approx 0.33$ (top of rep, chin over bar) and $y \approx 0.45$ (hanging), while the wrist landmarks (green/red) remain almost stationary at $y \approx 0.33$, confirming the grip is fixed on the bar. Each oscillation cycle corresponds to one pullup repetition. **Phase 3 (19.5–23.5 s):** session end—the shoulder Y-values drop sharply toward 0 as the user steps away and the phone is moved, breaking the framing.

The repetition counting logic in Stand Mode operates on the vertical displacement of the shoulder and wrist landmarks. As shown in

Fig. 2, the Y-coordinate of the shoulder landmark oscillates synchronously with the exercise rhythm. The `usePoseLandmarker` hook provides per-frame landmark arrays to a dedicated `ExerciseTracker` module, which implements the same dual-burst state machine as the inertial tracker but driven by landmark position deltas rather than energy.

5.3 Channel 3: Voice Command Navigation

Motivation. Even between exercises—when the user is not actively performing reps—touching the screen remains undesirable. Hands may be sweaty or occupied, and the disruption of switching mental focus from the exercise to a UI element is itself a source of friction.

The `useVoiceCommands` hook uses the Web Speech API’s `SpeechRecognition` interface in **continuous mode** (i.e., the recognition session does not terminate after a single utterance). It recognizes the following Italian commands:

Spoken command	Action
“Vai” / “Go”	Start / advance to next set
“Pausa”	Pause active timer
“Riprendi” / “Continua”	Resume paused timer
“Stop” / “Fermo”	End the current session

Table 2: Voice commands recognized by the application.

To prevent resource exhaustion (a known issue with persistent `SpeechRecognition` sessions on mobile browsers), the `onend` callback restarts the recognition engine with a 2-second delay. The recognition state is managed through a `React ref` rather than state to avoid re-triggering the `useEffect` on every render.

Challenge: mobile browser policy. iOS Safari blocks both the `AudioContext` and `SpeechSynthesis` from initializing unless they are invoked within the synchronous call stack of a direct user gesture (tap or click). Our initial implementation initialized these APIs at module load time, resulting in completely silent sessions on iOS. The solution was strict **lazy loading**: the `AudioContext` is created and resumed inside the `onClick` handler of the “Start Session” button. From that moment, the context remains active for the duration of the session, and all subsequent audio—both speech synthesis and beep tones—is reliably delivered.

5.4 Channel 4: Audio-Haptic State Communication

A key design goal is that the user should always be aware of the system’s state without looking at the screen. *No-Excuses* achieves this through a layered audio-haptic feedback system implemented entirely via the Web Audio API:

- **Session start signal:** A three-tone ascending arpeggio (440 Hz → 550 Hz → 660 Hz) accompanied by a triple vibration pattern [150, 80, 150, 80, 300]. This fires at the exact moment the countdown ends and counting begins, so the user knows to start exercising.
- **Repetition beep:** A short 880 Hz beep with a double vibration [60, 40, 60] fires immediately upon each confirmed repetition. This confirms to the user that their movement was registered without requiring any visual verification.
- **Speech count:** After each repetition, the `SpeechSynthesis` API announces the count aloud (“One”, “Two”, etc.), providing an additional, unmistakable confirmation.
- **Goal reached signal:** When the user-defined target repetition count is achieved, the same triple arpeggio as the start signal fires, and the accelerometer listener is immediately deactivated via `stopSession()`, preventing any subsequent false positives from device handling.
- **Rest timer announcements:** During the guided workout runner, the rest countdown announces intervals aloud (“Rest for thirty seconds”, countdown at 10 s, etc.) so the user can continue resting without watching the screen.

All tones are synthesized in real-time using the Web Audio API’s oscillator-gain graph, with exponential gain ramps to avoid clipping artifacts. This approach has zero dependency on audio files and guarantees playback latency < 10 ms.

6 Existing Tools and Libraries

- **React 18 + TypeScript + Vite:** SPA framework with strict type safety and fast HMR during development.

- **Supabase:** Open-source Firebase alternative providing PostgreSQL persistence and authentication for workout history.
- **Google MediaPipe Pose (1):** WebAssembly-compiled pose estimation pipeline providing 33 landmarks at 20–30 FPS on mid-range mobile devices.
- **Web Speech API:** Browser-native speech recognition (`SpeechRecognition`) and synthesis (`SpeechSynthesis`).
- **DeviceMotion / DeviceOrientation APIs:** Browser APIs for raw accelerometer (with gravity) and gyroscope access.
- **Web Audio API:** Low-latency audio synthesis; used for all workout sounds.
- **Navigator.vibrate():** Haptic feedback on Android devices.

7 Evaluation

7.1 Evaluation Protocol

Following the methodology of related HCI work, our evaluation combines **lab tests** (performed by the developers to validate the algorithm) and **field tests** (performed in real workout environments).

The primary metric for inertial tracking is **Absolute Counting Error** ($ACE = |\hat{n} - n|$) where $\hat{n}$ is the system count and n is the ground-truth count. We also track the number of **false positives** (extra counts generated by device handling) separately, since they are the most disruptive error type from a user experience standpoint.

7.2 Comparative Results: Pushup Pocket Mode

We compare three algorithm configurations across multiple pushup sets; full results are reported in Table 3 at the end of this document.

The naive system with a 2000 ms cooldown missed rapid reps (< 2 s duration) and double-counted the post-extraction spike. The intermediate configuration (peak-count without orientation filter) correctly handled rep tempo but still produced extraction false positives. Our final system ($\theta_{\max} = 66^\circ$, 1200 ms cooldown) achieved zero extraction false positives and perfect counting accuracy across all tested configurations.

7.3 Voice Command Recognition

Voice commands were tested across 20 sessions by the two developers. Recognition latency on an iPhone 14 (iOS 17, Safari) averaged ≈ 800 ms from utterance completion to action execution. All four command categories achieved $> 90\%$ recognition accuracy in a quiet indoor environment. In noisy gym environments (ambient music, weights), accuracy dropped to $\approx 75\%$, which remains within acceptable usability bounds for navigation tasks (the cost of an unrecognized command is that the user must repeat it—not an incorrect action).

7.4 Discussion

The most impactful improvement was the gravity orientation filter: it transformed an unreliable prototype into a deployable tool by eliminating the entire class of post-exercise false positives. The key insight is *geometric*: exercise motion in pocket mode is characterized by a stable device orientation (the phone stays in the same angular reference frame as the user’s thigh), while the device extraction motion necessarily involves a large orientation change.

The 1200 ms cooldown is a hard constraint on the maximum exercise tempo the system can support (one rep per 1.2 s). For highly trained athletes performing explosive reps faster than this rate, the system would undercount. This is a fundamental limitation of threshold-based counting and could be addressed in future work by learning a user-specific tempo profile.

A separate challenge, documented here for reproducibility, was the iOS AudioContext policy. This was not a correctness bug but a platform compatibility issue that rendered the entire audio-haptic communication channel silent for an entire class of devices. The lazy-loading solution is straightforward once the root cause is known, but it is not obvious from the Web Audio API documentation and cost significant debugging time.

8 Conclusion and Future Work

No-Excuses demonstrates that a multimodal approach combining inertial sensor fusion, computer vision, voice control, and audio-haptic feedback can replace manual workout tracking without proprietary hardware. Each modality contributes to a different part of the interaction: inertial sensors handle pocket-mode counting, computer vi-

sion handles standing exercises, voice commands eliminate inter-set screen interaction, and audio-haptic signals close the feedback loop without requiring visual attention.

The engineering decisions described—sensor fusion energy metric, peak-count algorithm, gravity orientation filter, lazy AudioContext initialization—collectively convert a straightforward threshold-based detector into a robust field-deployable system. The calibration protocol (20–30 in-the-wild sessions with varied rep pacing) was essential and could not be replaced by synthetic simulation.

Future Work. Planned extensions include: (1) expansion of the MediaPipe exercise library to squats, deadlifts, and dips; (2) integration of the onboard barometer for elevation tracking during outdoor calisthenics; (3) statistical dashboards showing workout volume and progression over time; and (4) exploration of on-device LLMs (e.g., running locally via WebLLM) to parse natural language voice commands such as “remove pullups and add dips”.

Appendix: Comparative Evaluation Results

System configuration	Ground Truth	Counted	FP
Naive threshold, 2000 ms cooldown, no gravity filter	6	8	2
Peak-count, 2000 ms cooldown, no gravity filter	5	4	2
Peak-count, 1200 ms, gravity filter (ours)	5	5	0

Table 3: Pushup repetition counting accuracy in Pocket Mode. *Ground Truth* = actual repetitions performed. *Counted* = repetitions registered by the system. *FP* = false positives generated by extracting the phone from the pocket at the end of the set. Results discussed in Section 7.2.

References

- [1] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Ting Lu, Drew George, Mogan Grundmann, Matthias Grundmann. Medi-aPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*, 2019. 5, 6
- [2] W3C Working Group. DeviceOrientation Event Specification, Level 2. <https://www.w3.org/TR/orientation-event/>, 2023.
- [3] W3C Community Group. Web Speech API. <https://wicg.github.io/speech-api/>, 2023.
- [4] Paul Adenot, Hongchan Choi. Web Audio API. W3C Recommendation. <https://www.w3.org/TR/webaudio/>, 2021.
- [5] Marcin Grzeszczak et al. Wrist-worn accelerometer-based repetition counting for resistance training. *Sensors*, 21(4):1467, 2021.